Design Pattern Session State e Serialized LOB

Sessione

Una sessione è definita come un'interazione a lungo termine tra un client e un server.

- **Durata**: tipicamente inizia quando un utente effettua il login e termina quando esegue il logout o semplicemente abbandona l'applicazione.
- **Struttura**: può consistere in una singola richiesta o in una serie di richieste consecutive.
- **Tipologie**: le sessioni (e i server) si distinguono in:
  - **stateless** (senza stato): il server non conserva alcuno stato tra una richiesta e l'altra. In questo caso, lo stato dell'oggetto serve solo per elaborare la risposta corrente e viene reimpostato alla richiesta successiva.
  - **stateful** (con stato): il server tiene traccia delle attività del client (es. le pagine visitate da un certo indirizzo IP) mantenendo queste informazioni tra le varie richieste.

Stato del server

Questo concetto riguarda l'impatto della gestione dello stato sulle risorse del server.

- **Consumo delle risorse**: un server stateful utilizza molte più risorse di uno stateless, perché deve mantenere in vita uno specifico oggetto server per servire tutte le richieste dello stesso utente.
- **Esempio di inefficienza**: se ci sono 100 utenti, servono 100 oggetti server mantenuti in memoria. Se un utente invia una richiesta ogni 10 secondi e l'elaborazione dura solo 1 secondo, l'oggetto server rimane inattivo in memoria per il 90% del tempo, sprecando risorse inutilizzate.
- **Vantaggio stateless**: in un server stateless, non importa quale oggetto serve la richiesta, permettendo un utilizzo più efficiente delle risorse.

Stato della sessione

Lo stato della sessione rappresenta i dati specifici relativi all'interazione corrente di un utente.

- **Distinzione**: i dati di sessione sono distinti dai "dati permanenti". I dati di sessione sono rilevanti solo per la sessione in corso (ad esempio, il contenuto di un carrello della spesa), mentre i dati permanenti devono essere disponibili per tutte le sessioni. Tuttavia, alcuni dati di sessione possono essere promossi a dati permanenti (ad esempio, quando si conferma l'acquisto).
- **Problema**: sebbene per efficienza si preferiscano server stateless (specialmente con molti utenti), le funzionalità come il carrello richiedono di mantenere uno stato.
- **Conservazione**: esistono tre scelte architetturali di base per memorizzare lo stato della sessione:
  - sul **client**.
  - sul **server**.
  - sul **database**.

Design Pattern Client Session State

Intento

L'intento di questo pattern è conservare lo stato della sessione direttamente sul client.

Funzionamento

Invece di memorizzare i dati sul server, il client possiede tutti i dati della sessione.

→ ad ogni **richiesta**, il client invia tutti i dati di sessione al server.

→ ad ogni **risposta**, il server rimanda indietro tutto lo stato aggiornato al client.

→ con un "rich-client", per tenere questi dati si utilizza spesso un Data Transfer Object (DTO)

Modalità di implementazione

Esistono principalmente tre modi per gestire questo stato con interfacce web:

1. **Parametri URL**: utile per pochi dati (come ad esempio un ID), ma la dimensione è limitata e può creare problemi con i segnalibri (bookmark).
2. **Campi nascosti**: si usa il tag `<input type="hidden">`. Lo stato viene serializzato (spesso in XML) e inserito in questi campi che viaggiano con il form ma non sono visibili all'utente.
3. **Cookie**: vengono inviati automaticamente avanti e indietro. Lo stato è serializzato nel cookie. Tuttavia, funzionano solo per un singolo dominio (siti separati in diversi nomi di dominio non vedono i cookie di altri siti) e l'utente potrebbe disattivarli.

Vantaggi e svantaggi

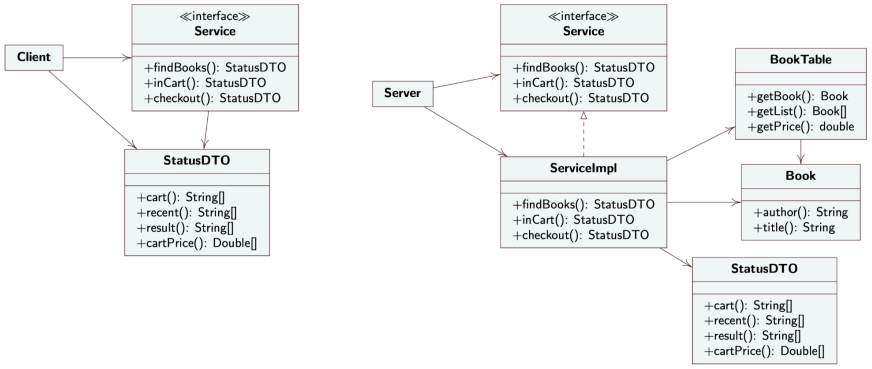
- **Vantaggi**: permette di avere server object stateless (senza stato), facilitando il recupero (failover) se un server si blocca, poiché il server successivo può riprendere la sessione dai dati inviati dal client.
- **Svantaggi**:
  - Se la quantità di dati è grande, i tempi di trasferimento aumentano drasticamente.
  - Sicurezza: i dati sul client possono essere visti e alterati. È necessario quindi criptare e convalidare i dati ad ogni richiesta, consumando risorse.

Esempio (e-commerce di libri)

Struttura e partecipanti

1. **StatusDTO:** è il componente chiave che funge da *Client Session State*. È un oggetto trasferibile che contiene tutto lo stato della sessione:
   - cart(): il carrello della spesa.
   - recent(): elementi visualizzati di recente.
   - result(): risultati di ricerca.
   - cartPrice(): prezzi del carrello.
2. **Lato client**: possiede il Client e utilizza l'interfaccia Service per comunicare. Mantiene lo StatusDTO.
3. **Lato server**:

- **Service** (interfaccia) e **ServiceImpl** (implementazione): gestiscono la logica (findBook, inCart, checkout). Notare che questi metodi ricevono e restituiscono lo **StatusDTO** per mantenere l'aggiornamento dello stato tra le chiamate senza salvarlo sul server.
- **BookTable** e **Book**: rappresentano i dati persistenti (il catalogo dei libri).



Design Pattern Server Session State

Intento

L'intento di questo pattern è conservare lo stato della sessione direttamente sul server in forma serializzata.

Funzionamento

- **In memoria**: gli oggetti che rappresentano la sessione vengono mantenuti nella memoria del server. Si utilizza una mappa che associa un ID di sessione (chiave) al riferimento dell'oggetto di sessione corrispondente. Quando arriva una richiesta con un ID, il server recupera l'oggetto dalla mappa per servirla.
- **Passivazione**: per gestire limiti di memoria (o per persistenza), lo stato dell'oggetto di sessione può essere serializzato (in formato binario o XML) e salvato su uno storage (file system o database locale). Questo processo è simile al pattern Memento.

Architetture avanzate

Se lo storage utilizzato per salvare lo stato serializzato è accessibile da più host, il pattern abilita due funzionalità critiche per sistemi distribuiti:

1. **Clustering**: le richieste possono essere distribuite su più server diversi.
2. **Failover**: se un server si blocca (va giù), un altro server può recuperare lo stato salvato e continuare la sessione senza interruzioni per l'utente.

Si può aver bisogno del pattern Serialized LOB per tenere lo stato.

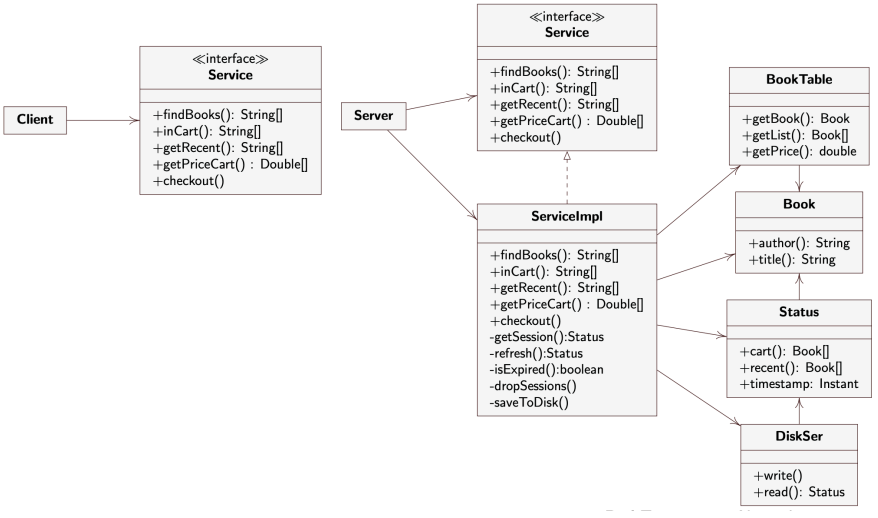
Gestione

Quando si memorizza lo stato su un database o file system, è necessario implementare un meccanismo di pulizia (spesso un *daemon* che controlla i timestamp) per cancellare i dati delle sessioni scadute o abbandonate.

Esempio

Struttura e partecipanti

1. **Lato server**:
   - **ServiceImpl**: è il componente centrale che gestisce la logica. Include metodi privati specifici per la gestione del ciclo di vita della sessione server-side:
     - getSession(): recupera la sessione attiva.
     - refresh(): aggiorna il timestamp della sessione.
     - isExpired(): controlla se la sessione è scaduta.
     - saveToDisk(): si occupa di persistere lo stato (passivazione).
     - dropSessions(): pulisce le sessioni vecchie.
   - **Status**: rappresenta l'oggetto dello stato della sessione vero e proprio. Contiene i dati specifici dell'utente:
     - cart: il carrello (array di Book).
     - recent: libri visualizzati di recente.
     - timestamp: l'istante dell'ultima interazione (per gestire la scadenza della sessione).
   - **DiskSer**: una classe di utilità responsabile delle operazioni di I/O per serializzare (write) e deserializzare (read) l'oggetto Status su disco.
   - **BookTable** / **Book**: rappresenta il catalogo dei dati persistenti.
2. **Lato client**:
   - Utilizza l'interfaccia **Service** per invocare i metodi remoti (findBook, inCart, checkout), ma non gestisce lo stato logicamente. Si limita a passare l'identificativo di sessione (implicito o esplicito) affinché il server possa recuperare lo **Status** corretto dalla memora o da disco.



Design Pattern Database Session State

Intento

L'intento di questo pattern è conservare lo stato della sessione (i dati specifici dell'interazione corrente) sotto forma di dati dedicati all'interno di un database.

Funzionamento

- **Recupero**: quando il server riceve una richiesta dal client, utilizza un ID di sessione (conservato sul client) come chiave per recuperare i dati necessari direttamente dal database.
- **Isolamento**: i dati recuperati (dati di sessione) devono essere trattati come dati locali e temporanei. Altre parti del sistema non dovrebbero utilizzarli finché la sessione non viene confermata (commit).

Strategie di memorizzazione (dati pending)

Poiché i dati di sessione sono dati "in sospeso" (cioè "pending") e non ancora confermati, esistono due strategie per distinguerli dai dati permanenti nel database:

1. **Modifica delle tabelle esistenti**: si aggiunge un campo alle tabelle reali (es. isPending booleano o un campo contenente l'ID di sessione) per marcare le righe come temporanee.
    - *Svantaggio*: è una soluzione invasiva, poiché costringe altre applicazioni che usano il database a dover gestire e filtrare questi campi extra.
2. **Tabelle separate** (strategia consigliata): si crea un set di tabelle dedicate solo ai dati di sessione. Spesso queste tabelle sono cloni di quelle reali con l'aggiunta dell'ID di sessione.
    - *Vantaggio*: rimuove l'invasività e facilita la gestione delle regole di integrità. I dati confermati hanno regole di validazione rigide, mentre i dati di sessione possono averne di diverse. Avere tabelle separate rende semplice applicare regole diverse.

Gestione del ciclo di vita (pulizia)

È necessario gestire la rimozione dei dati quando le sessioni terminano:

→ cancellazione esplicita: se una sessione viene chiusa, l'ID di sessione permette di trovare e cancellare tutti i dati associati.

→ timeout (sessioni abbandonate): se l'utente abbandona la sessione senza logout, i dati rimarrebbero nel DB. Per risolvere questo problema, si usa un processo in background (detto *daemon*) che controlla periodicamente i timestamp dell'ultima interazione e cancella i dati delle sessioni scadute.

Vantaggi e svantaggi

- **Vantaggi**:
    - permette di avere oggetti server **stateless** (senza stato in memoria), abilitando il pooling delle risorse.
    - facilita enormemente il **clustering** e il **failover**: se un server si blocca, un altro server può recuperare immediatamente lo stato del database e continuare la sessione.
    - semplifica la programmazione della gestione dello stato rispetto alla gestione in memoria.
- **Svantaggi**:
    - introduce una latenza maggiore, poiché è necessario accedere al database per ogni singola richiesta (problema che può essere mitigato dall'uso di una cache).

Design Pattern Serialized LOB

Intento

L'intento di questo pattern è conservare un intero **grafo di oggetti** serializzandoli all'interno di un singolo grande oggetto (LOB, *Large Object*), che viene poi inserito in un unico campo di un database.

Il **problema** nasce dal fatto che i sistemi a oggetti sono composti da molti piccoli oggetti interconnessi (es. una gerarchia di organizzazione o dettagli di prodotti). Mappare questa struttura in un database relazionale è complesso. Questo perché lo schema base richiederebbe molte tabelle collegate da chiavi esterne, costringendo a fare numerose join che rallentano le prestazioni e sono difficili da gestire.

Funzionamento

Invece di usare righe e relazioni, si usa la **serializzazione**. Tutto il grafo degli oggetti viene trasformato in una sequenza di byte e salvato in una tabella. La serializzazione può avvenire in due formati principali:

- **BLOB** (Binary Large Object): serializzazione binaria.
- **CLOB** (Character Large Object): serializzazione come caratteri.

L'oggetto serializzato contiene sia i dati che le informazioni sul tipo dell'oggetto stesso.

Problema della duplicazione

L'uso del Serialized LOB richiede attenzione alla duplicazione dei dati.

→ *esempio*: se si serializzano i dettagli di un "Cliente" direttamente dentro l'oggetto "Ordine" (LOB), i dati del cliente vengono ricopiati per ogni ordine.

→ per evitare ciò, il LOB potrebbe essere messo nella tabella "clienti" e gli "ordini" potrebbero avere solo un link ad essa, ma in generale, con questo pattern, una parte dei dati rischia spesso di venire duplicata.

Serializable

In questa sezione vengono discusse le avvertenze tecniche per implementare la serializzazione, necessaria per il pattern Serialized LOB e per salvare lo stato della sessione.

- **Requisito fondamentale**: quando si avvia la scrittura di un grafo di oggetti (ad esempio, tramite writeObject() di ObjectOutputStream), se si incontra un riferimento a un oggetto la cui classe non implementa l'interfaccia Serializable, si verificherà un errore a runtime.
- **Esclusione dei campi** (transient): se si vuole escludere un campo specifico dalla memorizzazione, questo deve essere dichiarato come **transient**.
    - *Vantaggio*: se un campo che contiene un riferimento a un oggetto è dichiarato transient, la classe di quell'oggetto referenziato **non** ha bisogno di essere Serializable, poiché quel campo verrà ignorato durante il processo.

È possibile vedere un esempio al seguente link: https://github.com/isd-inf/serialised-large-object/.